

SCX

State-Conditioned eXpertise —

2026-06-25 | v0.1.0-draft

1. 1
 2. 2
 3. 3
 4. 4
 5. 5
 6. 6SCX-Compress
 7. 7
 8. 8 Paper 1-3
 - 9.
-

1 (State Discovery & Representation)

1.1

$\mathcal{X} \ni \phi : \mathcal{X} \rightarrow \mathbb{R}^d \quad K \quad S = \{S_1, S_2, \dots, S_K\} /$

1.2

1.2.1 $\phi : \mathcal{X} \rightarrow \mathbb{R}^d \quad d$

- **ACE (Atomic Cluster Expansion)** d bispectrum $d \sim O(N_{\max}^3) \quad N_{\max}$
- **SOAP (Smooth Overlap of Atomic Positions)** $d \sim n_{\max} \times l_{\max}$
- **embedding** $h(x) = \text{Encoder}(x; \theta) \in \mathbb{R}^d$
- **CLIP embedding** $d = 512 \quad 768$

1.2.2 $\phi(x) \in \mathbb{R}^d \quad \text{ACE} \quad p \ll d$

PCA (Principal Component Analysis)

$U \in \mathbb{R}^{d \times p}$

$$U^* = \arg \max_{U: U^\top U = I} \text{Tr}(U^\top \Sigma U)$$

$$\Sigma = \frac{1}{N} \sum_{i=1}^N (\phi(x_i) - \bar{\phi})(\phi(x_i) - \bar{\phi})^\top$$

$$\tilde{\phi}(x) = U^\top \phi(x) \in \mathbb{R}^p$$

UMAP (Uniform Manifold Approximation and Projection)

$$k\text{-NN } G = (V, E, w) \quad w_{ij} = \exp(-(d_{ij} - \rho_i)/\sigma_i)$$

$$L_{\text{UMAP}} = \sum_{(i,j) \in E} \left[w_{ij} \log \left(\frac{w_{ij}}{v_{ij}} \right) + (1 - w_{ij}) \log \left(\frac{1 - w_{ij}}{1 - v_{ij}} \right) \right]$$

$$v_{ij} = (1 + a \cdot \|\tilde{\phi}(x_i) - \tilde{\phi}(x_j)\|^{2b})^{-1}$$

PaCMAP (Pairwise Controlled Manifold Approximation)

$$L_{\text{PaCMAP}} = \sum_{(i,j) \in \text{FP}} w_{ij} \cdot d_{ij}^2 + \sum_{(i,j) \in \text{MN}} \max(0, 1 - d_{ij}) + \sum_{(i,j) \in \text{FN}} \max(0, 1 - d_{ij})$$

$$\text{FP} = \text{MN} = \text{FN} =$$

1.2.3

$$x \ s(x) \in \{1, \dots, K\}$$

- K-means: $s(x) = \arg \min_k \|\tilde{\phi}(x) - \mu_k\|^2$
- : $s(x) = \arg \min_k$ distance in spectral embedding space

$$\gamma_s(x) = P(s|x) \sum_{s=1}^K \gamma_s(x) = 1$$

(GMM)

$$\gamma_s(x) = \frac{\pi_s \cdot \mathcal{N}(\tilde{\phi}(x); \mu_s, \Sigma_s)}{\sum_{t=1}^K \pi_t \cdot \mathcal{N}(\tilde{\phi}(x); \mu_t, \Sigma_t)}$$

$$\{\pi_s, \mu_s, \Sigma_s\}_{s=1}^K \text{ EM}$$

E-step

$$\gamma_s(x_i) = \frac{\pi_s \cdot \mathcal{N}(\tilde{\phi}(x_i); \mu_s, \Sigma_s)}{\sum_t \pi_t \cdot \mathcal{N}(\tilde{\phi}(x_i); \mu_t, \Sigma_t)}$$

M-step

$$N_s = \sum_i \gamma_s(x_i), \quad \pi_s = \frac{N_s}{N}, \quad \mu_s = \frac{1}{N_s} \sum_i \gamma_s(x_i) \tilde{\phi}(x_i)$$

$$\Sigma_s = \frac{1}{N_s} \sum_i \gamma_s(x_i) (\tilde{\phi}(x_i) - \mu_s)(\tilde{\phi}(x_i) - \mu_s)^\top$$

$$P(x \in S_s) = \mathbb{E}_{x \sim P(\cdot|s)}[\mathbf{1}_{S_s}(x)] \approx \frac{1}{N_s} \sum_{i=1}^N \gamma_s(x_i)$$

1.2.4 K

- **(BIC)** $\text{BIC}(K) = -2 \log \mathcal{L}(K) + p_K \log N$ BIC K
- **(Silhouette Score)** $S(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$ $a(i)$ $b(i)$
- bootstrap K

1.3

Input:

- X: ndarray, shape=(N, d_X),
- phi_func: callable, X -> R^d,
- n_states: int, K ()
- dim_red: str, : 'pca'/'umap'/'pacmap'
- n_components: int, p,

Output:

- states: list[State], K
- State :
- id: int,
 - center: ndarray, shape=(p,), (GMM mean)
 - cov: ndarray, shape=(p,p), (GMM cov)
 - weight: float, pi_s

- members: list[int],
- gamma: ndarray, shape=(N,K),
- phi_X: ndarray, shape=(N,d),
- phi_tilde: ndarray, shape=(N,p),
- reducer: object, (transform)
- clusterer: object, (predict)

1.4

1

Algorithm: StateDiscovery(X, phi_func, n_states, method='soft')

Step 1:

```
Phi_X = phi_func(X) # X -> R^d
```

Step 2: ()

```
if dim_red == 'pca':
    Phi_tilde = PCA(Phi_X, n_components=p)
elif dim_red == 'umap':
    Phi_tilde = UMAP(Phi_X, n_components=p)
elif dim_red == 'pacmap':
    Phi_tilde = PaCMAP(Phi_X, n_components=p)
```

Step 3:

```
if method == 'soft':
    gmm = GMM(n_components=n_states)
    gamma = gmm.fit_predict_proba(Phi_tilde)
elif method == 'hard':
    kmeans = KMeans(n_clusters=n_states)
    labels = kmeans.fit_predict(Phi_tilde)
    gamma = one_hot(labels, n_states)
```

Step 4:

```
for s in 1..K:
    rho[s] = mean(gamma[:, s]) #
    for each property p:
        mean_p[s] = weighted_mean(X_property, gamma[:, s])
```

Result: {states, gamma, phi_X, phi_tilde}

1.5

2 ()	$\gamma_s(x), \rho(s)$	
3 ()	$\gamma_s(x), \rho(s),$	C(s), N(s), D(s)
5 ()	$\gamma_s(x)$	
7 ()	$\rho(s),$	
6 ()	$\gamma_s(x),$	anchor

1.6

$\phi(X)$	$O(N \cdot C_\phi)$	$O(N \cdot d)$
PCA	$O(Nd^2 + d^3)$	$O(d^2)$
UMAP	$O(N \log N \cdot p)$	$O(N \cdot k)$
GMM (EM)	$O(NKp^2 \cdot T)$	$O(NK + Kp^2)$
K-means	$O(NKp \cdot T)$	$O(NK)$

C_ϕ ϕ T k UMAP

1.7

	d		
	p	2-50	2-310-50
	K		BIC/
	-	soft	soft (GMM) /
	-	PCA	hard (K-means)
			PCA/UMAP/PaCMAP
			UMAP
GMM	-	full	full/diag/tied/spherical

2 (Expert Reliability Estimation)

2.1

$M \{f_1, f_2, \dots, f_M\} f_m : \mathcal{X} \rightarrow \mathcal{Y} m s$

2.2

2.2.1 (Supervised) $\mathcal{D}_L = \{(x_i, y_i)\}_{i=1}^{N_L} y_i = f^*(x_i)$ oracle DFT

1 (State-Conditioned Expert Risk)

$$R_m(s) = \mathbb{E}_{x \sim P(\cdot|s)}[\ell(f_m(x), f^*(x))]$$

$$\hat{R}_m(s) = \frac{\sum_{i=1}^{N_L} \gamma_s(x_i) \cdot \ell(f_m(x_i), y_i)}{\sum_{i=1}^{N_L} \gamma_s(x_i)}$$

$\gamma_s(x) 1$

2SCX (SCX Reliability)

$$\text{SCX}_m(s) = \mathbb{P}(\ell(f_m(x), y) < \tau \mid x \in s)$$

$m s \tau$

$$\widehat{\text{SCX}}_m(s) = \frac{\sum_{i=1}^{N_L} \gamma_s(x_i) \cdot \mathbf{1}[\ell(f_m(x_i), y_i) < \tau]}{\sum_{i=1}^{N_L} \gamma_s(x_i)}$$

$$\mathbb{V}[\hat{R}_m(s)] \approx \frac{1}{N_s} \cdot \frac{1}{N_s - 1} \sum_{i=1}^{N_L} \gamma_s(x_i) \left(\ell(f_m(x_i), y_i) - \hat{R}_m(s) \right)^2$$

$N_s = \sum_i \gamma_s(x_i) s$

2.2.2 (Unsupervised)

$$\text{Disagreement}_m(s) = \frac{1}{M-1} \sum_{n \neq m} \mathbb{E}_{x \sim P(\cdot|s)} [\ell(f_m(x), f_n(x))]$$

$$\hat{D}_m(s) = \frac{1}{M-1} \sum_{n \neq m} \frac{\sum_i \gamma_s(x_i) \cdot \ell(f_m(x_i), f_n(x_i))}{\sum_i \gamma_s(x_i)}$$

SCX

$$\widehat{\text{SCX}}_m^{(\text{unsup})}(s) = 1 - \sigma \left(\frac{\hat{D}_m(s) - \mu_D}{\sigma_D} \right)$$

$\sigma(\cdot)$ sigmoid μ_D, σ_D (m, s)

2.2.3 (Few-shot) $s \ N_s < N_{\min}$

A (Bayesian Smoothing)

$$\hat{R}_m^{(\text{Bayes})}(s) = \frac{N_s \cdot \hat{R}_m(s) + \lambda \cdot \hat{R}_m(\cdot)}{N_s + \lambda}$$

$$\hat{R}_m(\cdot) = \frac{1}{K} \sum_{s'} \hat{R}_m(s') \quad m \ \lambda$$

B (Hierarchical Bayesian)

$$R_m(s) \sim \mathcal{N}(\mu_m, \tau_m^2) \quad ()$$

$$\ell_{mi} \mid R_m(s) \sim \mathcal{N}(R_m(s), \sigma_m^2/w_{si}) \quad ()$$

$$w_{si} = \gamma_s(x_i)$$

$$\hat{R}_m^{(\text{HB})}(s) = \frac{\frac{N_s}{\sigma_m^2} \cdot \hat{R}_m(s) + \frac{1}{\tau_m^2} \cdot \mu_m}{\frac{N_s}{\sigma_m^2} + \frac{1}{\tau_m^2}}$$

C (Transfer Estimation)

$$\hat{R}_m^{(\text{Transfer})}(s) = \frac{\sum_{s'} \kappa(\mu_s, \mu_{s'}) \cdot N_{s'} \cdot \hat{R}_m(s')}{\sum_{s'} \kappa(\mu_s, \mu_{s'}) \cdot N_{s'}}$$

$$\kappa(\mu_s, \mu_{s'}) = \exp(-\beta \|\mu_s - \mu_{s'}\|^2)$$

2.3

Input:

- experts: list[callable], M f_m: X -> Y
- X_labeled: ndarray, shape=(N_L, d_X),
- y_labeled: ndarray, shape=(N_L,),
- gamma: ndarray, shape=(N_L, K), (1)
- loss_fn: callable, ell(y_pred, y_true)
- tau: float, SCX (0.1)
- method: str, 'supervised'/'unsupervised'/'bayesian'/'hierarchical'
- lambda_smooth: float, (5.0)

Output:

- R: ndarray, shape=(M, K),
- R_var: ndarray, shape=(M, K),
- SCX: ndarray, shape=(M, K), SCX
- disagreement: ndarray, shape=(M, K), ()

2.4

2

Algorithm: EstimateExpertReliability(L, U, gamma, experts, method)

Phase 1:

```

for each expert m:
    for each sample i:
        loss_mi = loss_fn(f_m(x_i), y_i) #
        OR dis_mi = mean_{n!=m} loss(f_m(x_i), f_n(x_i)) #

```

Phase 2:

```

for each state s:
    N_s = sum_i gamma[i, s] + eps

```

```

    for each expert m:
        R[m, s] = sum_i gamma[i,s] * loss_mi / N_s
        SCX[m, s] = sum_i gamma[i,s] * I[loss_mi < tau] / N_s

Phase 3: ( N_s < N_min)
    for each state s where N_s < N_min:
        for each expert m:
            R[m, s] = bayesian_smooth(R[m, s], R[m, :], N_s, lambda)

Phase 4:
    R_var[m, s] = compute_variance(loss_mi, gamma[:,s], N_s)

Result: {R, R_var, SCX}

```

2.5

3 ()	$R_m(s), SCX_m(s)$	C(s), ExpertConflict
4 ()	$R_m(s), SCX_m(s)$	max_m SCX_m(s)
5 ()	$R_m(s), SCX_m(s)$	
7 ()	max_m SCX_m(s)	route

2.6

$O(M \cdot N \cdot C_f)$	$O(M \cdot N)$
$O(M \cdot N \cdot K)$	$O(M \cdot K)$
$O(M \cdot K)$	$O(M \cdot K)$
$O(M^2 \cdot N \cdot C_f)$	$O(M \cdot N)$

C_f

2.7

SCX	τ	0.1	””
	λ	5.0	
	$N_{\min}$	10	
	β	1.0	
	τ_m^2	1.0	

3 (Data Four-Classification)

3.1

s ValuableRedundantNoisyExpert-dependent

3.2

3.2.1 (Learnability Score)

$$L(s) = C(s) \cdot [1 - N(s)]$$

$C(s) \in [0, 1]$ (Consistency) $N(s) \in [0, 1]$ (Noise Score)

- $L(s) \rightarrow 1$
- $L(s) \rightarrow 0$

3.2.2 (Consistency Measures) $C(s)$

A (Label Consistency)

$$C_{\text{label}}(s) = \frac{1}{\binom{M}{2}} \sum_{m < n} \mathbb{E}_{x \sim P(\cdot|s)} [\mathbf{1}[f_m(x) = f_n(x)]]$$

oracle

$$C_{\text{var}}(s) = \exp \left(-\frac{\text{Var}(y|s)}{\sigma_0^2} \right), \quad \text{Var}(y|s) = \frac{\sum_i \gamma_s(x_i)(y_i - \bar{y}_s)^2}{\sum_i \gamma_s(x_i)}$$

B (Expert Agreement Consistency)

$$C_{\text{expert}}(s) = 1 - \frac{1}{M} \sum_{m=1}^M \frac{\hat{D}_m(s)}{\max_{m'} \hat{D}_{m'}(s) + \varepsilon}$$

$\hat{D}_m(s)$ m s

C (Density Neighborhood Consistency)

/

$$C_{\text{density}}(s) = \frac{1}{|s|} \sum_{x_i \in s} \frac{1}{k} \sum_{x_j \in \text{NN}_k(x_i)} \mathbf{1}[\hat{y}_i \approx \hat{y}_j]$$

$\hat{y}_i$

$$C(s) = \alpha \cdot C_{\text{label}}(s) + \beta \cdot C_{\text{expert}}(s) + \gamma \cdot C_{\text{density}}(s)$$

$$\alpha + \beta + \gamma = 1$$

3.2.3 (Noise Score)

$$\text{NoiseScore}(x_i) = r_i \cdot \frac{1}{\rho(s_i) + \varepsilon} \cdot [1 - C(s_i)]$$

$$r_i = \ell(f(x_i), y_i)$$

- + + $\rightarrow$ NoiseScore $\rightarrow 0 \rightarrow$ - + + $\rightarrow$ NoiseScore $\rightarrow$ large $\rightarrow$ - $\rightarrow$ NoiseScore $\rightarrow 0 \rightarrow$

$$N(s) = \sigma \left(\frac{1}{|s|} \sum_{x_i \in s} \text{NoiseScore}(x_i) \right)$$

$\sigma(\cdot)$ sigmoid $N(s) \in [0, 1]$

3.2.4 (Redundancy Estimation) Definition $D(s) \in [0, 1]$ s

A

$$D_{\text{count}}(s) = 1 - \exp\left(-\frac{|\mathcal{D}_{\text{train}} \cap s|}{n_{\text{eff}}(s)}\right)$$

$n_{\text{eff}}(s)$ s

B

$$D_{\text{div}}(s) = 1 - \frac{H(\mathcal{D}_{\text{train}} \cap s)}{H(\mathcal{U} \cap s)}$$

$H(\cdot)$ $\mathcal{U}$

C

$$D_{\text{model}}(s) = \exp\left(-\frac{1}{|s|} \sum_{x_i \in s} \mathbb{H}[p(y|x_i; \theta)]\right)$$

$\mathbb{H}[\cdot]$ $D(s) \rightarrow 1$

$$D(s) = \min(1, D_{\text{count}}(s) + D_{\text{model}}(s))$$

3.2.5 (Boundary Detection) —

$$\text{BoundaryScore}(x) = 1 - (\gamma_{(1)}(x) - \gamma_{(2)}(x))$$

$\gamma_{(1)}(x)$ $\gamma_{(2)}(x)$

- $\text{BoundaryScore} \rightarrow 0$
- $\text{BoundaryScore} \rightarrow 1$

$$s_{\text{boundary}} = \{x \mid \text{BoundaryScore}(x) > \tau_{\text{boundary}}\}$$

6 7

3.2.6

Valuable	+ + +	$\bar{r}(s) > \tau_r, \rho(s) > \tau_\rho,$ $L(s) > \tau_L, D(s) < \tau_D$
Redundant	+	$\bar{r}(s) < \tau_r, D(s) > \tau_D$
Noisy	+ +	$\bar{r}(s) > \tau_r, \rho(s) < \tau_\rho,$ $C(s) < \tau_C$
Expert- dependent	+	$\max_m \text{SCX}_m(s) \gg$ $\text{SCX}_{\text{others}}(s)$

Noisy > Expert-dependent > Valuable > Redundant

3.3

Input:

- X: ndarray, shape=(N, d_X),
- y: ndarray, shape=(N,), (None)
- gamma: ndarray, shape=(N, K), (1)
- R: ndarray, shape=(M, K), (2)
- SCX: ndarray, shape=(M, K), SCX (2)
- experts: list[callable], M
- model: callable, f: X -> Y
- params: dict,

Output:

- classification: dict[state_id -> Category]
 {VALUABLE, REDUNDANT, NOISY, EXPERT_DEPENDENT}
- L: ndarray, shape=(K,),
- C: ndarray, shape=(K,),

- N: ndarray, shape=(K,),
- D: ndarray, shape=(K,),
- r_bar: ndarray, shape=(K,),
- rho: ndarray, shape=(K,),
- boundary_idx: list[int],
- NoiseScore: ndarray, shape=(N,),

3.4

3

Algorithm: FourClassify(gamma, R, SCX, X, y, params)

Step 1:

```

for s in 1..K:
    r_bar[s] = weighted_mean(r_i, gamma[:,s])    #
    rho[s] = mean(gamma[:,s])                  #

```

Step 2: C(s)

```

if labeled data available:
    C_label[s] = intra_state_variance(y, gamma[:,s])
else:
    C_expert[s] = 1 - normalized_disagreement(R[:,s])
C[s] = combine(C_label[s], C_expert[s])

```

Step 3: N(s)

```

for i in 1..N:
    s_i = argmax_s gamma[i,:]
    NoiseScore[i] = r_i * (1 / (rho[s_i] + eps)) * (1 - C[s_i])
N[s] = sigmoid(mean(NoiseScore[members_of_s]))

```

Step 4: L(s)

```

L[s] = C[s] * (1 - N[s])

```

Step 5: D(s)

```

D_count[s] = 1 - exp(-n_train[s] / n_eff[s])
D[s] = min(1, D_count[s])

```

Step 6:

```

for i in 1..N:
    gamma_sorted = sort(gamma[i,:], descending)
    boundary_score[i] = 1 - (gamma_sorted[0] - gamma_sorted[1])
    boundary_idx = where(boundary_score > tau_boundary)

```

Step 7:

```

for s in 1..K:
    if r_bar[s] > tau_r and rho[s] < tau_rho and C[s] < tau_C:
        class[s] = NOISY
    elif max(SCX[:,s]) - second_max(SCX[:,s]) > tau_SCX_gap:
        class[s] = EXPERT_DEPENDENT
    elif r_bar[s] > tau_r and rho[s] > tau_rho and L[s] > tau_L and D[s] < tau_D:
        class[s] = VALUABLE
    elif r_bar[s] < tau_r and D[s] > tau_D:
        class[s] = REDUNDANT
    else:
        class[s] = VALUABLE # default to valuable (conservative)

```

Result: {classification, L, C, N, D, boundary_idx}

3.5

1. **(Empty State)** $\rho(s) \approx 0$ REDUNDANT
2. **(Novel State)** $N_s < N_{\min}$ VALUABLE
3. **(High Conflict)** $\text{Conflict}(s) > \tau_{\text{conflict}}$ $C(s) > \tau_C$ EXPERT_DEPENDENT
4. **(Fully Noisy)** $L(s) < \varepsilon_L$ NOISY

3.6

4 ()	$L(s), C(s), N(s), D(s)$	$V_{\text{add}}(s)$
6 ()	$D(s), \text{boundary_idx}$	$V_{\text{remove}}(s)$
7 ()	classification, $L(s)$	/
5 ()	ExpertConflict(s)	

3.7

$O(N \cdot K)$	$O(K)$
$O(N \cdot M \cdot C_f + N \cdot K)$	$O(K)$
$O(N \cdot K)$	$O(N)$
$O(N \cdot K)$	$O(K)$
$O(N \cdot K \log K)$	$O(N)$

3.8

	τ_r	0.5 ()	/
	τ_ρ	0.05	/
	τ_C	0.3	/
	τ_L	0.4	/
	τ_D	0.7	/
SCX	$\tau_{\text{SCX_gap}}$	0.3	
	τ_{boundary}	0.3	
	α, β, γ	(0.5, 0.3, 0.2)	//

4 (State Data Valuation)

4.1

1. $V_{\text{add}}(s)$ s
2. $V_{\text{remove}}(s)$ s

4.2

4.2.1 (Acquisition Value)

$$V_{\text{add}}(s) = \bar{r}(s) \cdot \rho(s) \cdot L(s) \cdot [1 - D(s)] \cdot \max_m \text{SCX}_m(s)$$

$\bar{r}(s)$
$\rho(s)$
$L(s)$
$1 - D(s)$
$\max_m \text{SCX}_m(s)$

$$V_{\text{add}}(s) \text{ --- } \bar{r}(s) \cdot D(s)$$

$$V_{\text{add}}^{(\text{cost})}(s) = V_{\text{add}}(s) - \lambda_c \cdot \text{Cost}_{\text{acquire}}(s)$$

$$\text{Cost}_{\text{acquire}}(s) \text{ } s \text{ DFT}$$

4.2.2 (Compression Value) Definition

$$V_{\text{remove}}(s) = \bar{r}(s) \cdot D(s) \cdot (1 - \rho(s)) \cdot \frac{1}{1 + L(s)}$$

- $\bar{r}(s) + D(s) \text{ } s \rightarrow$
- $\rho(s)$
- $L(s)$

leave-one-state-out

$$V_{\text{remove}}(s) = \mathcal{L}(\theta; \mathcal{D} \setminus \mathcal{D}_s) - \mathcal{L}(\theta; \mathcal{D})$$

$$\mathcal{L}(\theta; \mathcal{D}) \text{ } \mathcal{D}_s \text{ } s$$

$$V_{\text{remove}}(s) \approx \frac{1}{|\mathcal{D}_s|} \sum_{x_i \in \mathcal{D}_s} \nabla_{\theta} \ell(x_i; \theta)^{\top} H_{\theta}^{-1} \nabla_{\theta} \mathcal{L}(\mathcal{D})$$

$$H_{\theta} \text{ Hessian}$$

4.2.3 Shapley Shapley i

$$\phi_i = \sum_{S \subseteq \mathcal{D} \setminus \{i\}} \frac{|S|!(|\mathcal{D}| - |S| - 1)!}{|\mathcal{D}|!} [v(S \cup \{i\}) - v(S)]$$

$v(S)$ S

SCX Shapley

SCX Shapley

$$V_{\text{add}}(s) \approx \frac{1}{|\mathcal{D}_s|} \sum_{i \in \mathcal{D}_s} \phi_i$$

- Shapley $O(2^{|\mathcal{D}|}) \rightarrow$ - SCX $O(K) \rightarrow$ - SCX

4.2.4 Influence Function i

$$\mathcal{I}(x_i) = -H_{\theta}^{-1} \nabla_{\theta} \ell(x_i; \theta)$$

SCX

$$V_{\text{remove}}(s) \propto \frac{1}{|\mathcal{D}_s|} \sum_{i \in \mathcal{D}_s} \|\mathcal{I}(x_i)\|$$

4.2.5

$$\text{Cost}_{\text{acquire}}(s) = c_0 \cdot \mathbb{E}_{x \sim P(\cdot|s)} [\text{Cost}_{\text{label}}(x)]$$

MLIP $\text{Cost}_{\text{label}}(x)$ DFT

$$U_{\text{acquire}}(s) = V_{\text{add}}(s) - \lambda \cdot \text{Cost}_{\text{acquire}}(s)$$

4.3

Input:

- `r_bar`: ndarray, shape=(K,), (3)
- `rho`: ndarray, shape=(K,), (1/3)
- `L`: ndarray, shape=(K,), (3)
- `D`: ndarray, shape=(K,), (3)
- `SCX`: ndarray, shape=(M, K), SCX (2)
- `cost_per_state`: ndarray, shape=(K,),
- `lambda_cost`: float,

Output:

- `V_add`: ndarray, shape=(K,),
- `V_remove`: ndarray, shape=(K,),
- `V_add_cost`: ndarray, shape=(K,),
- `ranking_add`: list[int], `V_add` ID
- `ranking_remove`: list[int], `V_remove` ID

4.4

4

Algorithm: `ComputeStateValues(r_bar, rho, L, D, SCX)`

Step 1:

```
for s in 1..K:
    best_expert = max(SCX[:, s])
    V_add[s] = r_bar[s] * rho[s] * L[s] * (1 - D[s]) * best_expert
```

Step 2:

```
for s in 1..K:
    V_remove[s] = r_bar[s] * D[s] * (1 - rho[s]) / (1 + L[s] + eps)
```

Step 3: ()

```
for s in 1..K:
    V_add_cost[s] = V_add[s] - lambda_cost * cost_per_state[s]
```

Step 4:

```
ranking_add = argsort(V_add, descending)
```

```
ranking_remove = argsort(V_remove, descending)
```

```
Result: {V_add, V_remove, V_add_cost, ranking_add, ranking_remove}
```

4.5

7 ()	$V_{\text{add}}(s), \text{ranking_add}$
6 ()	$V_{\text{remove}}(s), \text{ranking_remove}$
	ranking

4.6

$O(K \cdot M)$	$O(K)$
$O(K)$	$O(K)$
$O(K \log K)$	$O(K)$

4.7

λ	0.0	
-	true	$L(s) < \varepsilon_L$ $V_{\text{add}}(s) = 0$

5 (Expert Routing & Arbitration)

5.1

x

5.2

5.2.1 (Hard Routing)

$$m^*(x) = \arg \min_m \sum_{s=1}^K \gamma_s(x) \cdot \hat{R}_m(s)$$

x

$$m^*(x) = \arg \min_m \left[\sum_{s=1}^K \gamma_s(x) \cdot \hat{R}_m(s) + \lambda \cdot C_m \right]$$

C_m m GPU

SCX

$$m^*(x) = \arg \max_m \sum_{s=1}^K \gamma_s(x) \cdot \text{SCX}_m(s)$$

5.2.2 (Soft Weighting)

$$w_m(x) = \frac{\exp \left(-\alpha \cdot \sum_s \gamma_s(x) \cdot \hat{R}_m(s) \right)}{\sum_{n=1}^M \exp \left(-\alpha \cdot \sum_s \gamma_s(x) \cdot \hat{R}_n(s) \right)}$$

$\alpha \geq 0$ - $\alpha \rightarrow 0$ $w_m(x) \rightarrow 1/M$ - $\alpha \rightarrow \infty$ one-hot

SCX

$$w_m(x) = \frac{\sum_s \gamma_s(x) \cdot \text{SCX}_m(s)^\beta}{\sum_n \sum_s \gamma_s(x) \cdot \text{SCX}_n(s)^\beta}$$

$\beta \geq 1$

$$f_{\text{ensemble}}(x) = \sum_{m=1}^M w_m(x) \cdot f_m(x)$$

5.2.3 (Conflict Detection)

$$\text{Conflict}(s) = \frac{2}{M(M-1)} \sum_{m < n} \mathbb{E}_{x \sim P(\cdot|s)} [\ell(f_m(x), f_n(x))]$$

$$\text{Conflict}(s) = \frac{1}{M} \sum_{m=1}^M (\hat{R}_m(s) - \bar{R}(s))^2, \quad \bar{R}(s) = \frac{1}{M} \sum_m \hat{R}_m(s)$$

$$\text{Conflict}(x) = \frac{2}{M(M-1)} \sum_{m < n} \ell(f_m(x), f_n(x))$$

$$\text{Action}(x) = \begin{cases} \text{route_to_expert} & \text{if } \exists m : \text{SCX}_m(s(x)) \gg \text{SCX}_{\text{others}}(s(x)) \\ \text{request_relabel} & \text{if } \text{Conflict}(x) > \tau_{\text{conflict}} \text{ and oracle_available} \\ \text{weighted_ensemble} & \text{otherwise} \end{cases}$$

5.2.4 (Arbitration Strategies) A (Weighted Ensemble)

5.2.2

B (Best Expert)

$$f(x) = f_{m^*}(x), \quad m^* = \arg \min_m \sum_s \gamma_s(x) \cdot \hat{R}_m(s)$$

C (Confidence Filtering)

$$\text{SCX}_m(s(x)) > \tau_{\text{SCX}}$$

$$\mathcal{M}_{\text{conf}}(x) = \{m \mid \sum_s \gamma_s(x) \cdot \text{SCX}_m(s) > \tau_{\text{SCX}}\}$$

$$f(x) = \frac{1}{|\mathcal{M}_{\text{conf}}(x)|} \sum_{m \in \mathcal{M}_{\text{conf}}(x)} f_m(x)$$

D (Meta-Expert)

$$g : \mathbb{R}^M \rightarrow \mathcal{Y} [f_1(x), \dots, f_M(x)]$$

$$f(x) = g(f_1(x), f_2(x), \dots, f_M(x); \theta_g)$$

5.2.5 MoE MoE (Mixture of Experts)

$$f_{\text{MoE}}(x) = \sum_{m=1}^M g_m(x; \theta_g) \cdot f_m(x)$$

$$g_m(x; \theta_g) \text{ softmax}$$

SCX vs. MoE

MoE	SCX
/	

SCX MoE

5.3

Input:

- x: ndarray, shape=(batch, d_X),
- gamma_func: callable, x -> gamma, shape=(batch, K),
- R: ndarray, shape=(M, K), (2)
- SCX: ndarray, shape=(M, K), SCX (2)
- experts: list[callable], M


```

- alpha:          float,
- strategy:       str, 'hard'/'soft_weighted'/'confidence_filter'/'meta_expert'
- tau_SCX:        float,
- lambda_cost:    float,
- costs:          ndarray, shape=(M,),

```

Output:

```

- y_pred:         ndarray, shape=(batch,), /
- weights:        ndarray, shape=(batch, M),
- selected_expert: ndarray, shape=(batch,), ID
- conflict_score: ndarray, shape=(batch,),
- routing_info:    dict,

```

For soft weighting:

```

-  $w_m(x) \in [0,1]$ ,  $\sum_m w_m(x) = 1$ ,

```

5.4

5

Algorithm: RouteAndArbitrate(x, experts, R, SCX, gamma_func, strategy)

Step 1:

```

    gamma = gamma_func(x) # shape=(batch, K)

```

Step 2:

```

    for each expert m:
        weighted_R[m] = sum_s gamma * R[m, s] # shape=(batch,)

```

Step 3:

```

    if strategy == 'hard':
        selected = argmin_m weighted_R
        weights = one_hot(selected, M)

    elif strategy == 'soft_weighted':
        weights = softmax(-alpha * weighted_R, axis=-1)

    elif strategy == 'confidence_filter':

```

```

weighted_SCX[m] = sum_s gamma * SCX[m, s]
mask = weighted_SCX > tau_SCX
if all(mask == False):
    mask = True # fallback: use all experts
weights = mask / sum(mask)

```

Step 4:

```

predictions = stack([f_m(x) for f_m in experts]) # shape=(M, batch)
conflict = pairwise_disagreement(predictions)

```

Step 5:

```

y_pred = sum_m weights[:, :, m] * predictions[m, :, :]

() conflict > tau_conflict :
    arbitration_strategy(x, predictions, weights, conflict)

```

Result: {y_pred, weights, conflict}

5.5

- 1 () $\gamma_s(x)$
- 2 () $R_m(s), SCX_m(s)$
- 3 () ExpertConflict(s)

7 ()	conflict_score	relabel
------	----------------	---------

5.6

$O(\text{batch} \cdot K \cdot d_\phi)$	$O(\text{batch} \cdot K)$
$O(\text{batch} \cdot M \cdot K)$	$O(\text{batch} \cdot M)$
$O(\text{batch} \cdot M \cdot C_f)$	$O(\text{batch} \cdot M)$

$O(\text{batch} \cdot M^2)$	$O(\text{batch} \cdot M)$

5.7

	α	1.0	
SCX	τ_{SCX}	0.7	SCX
	τ_{conflict}	0.3	
	λ	0.0	
-		<u>weighted_ensemble</u>	

6SCX-Compress

6.1

$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ SCX-Compress $\mathcal{D}_c = \{(x_i, y_i, w_i)\}$ 1. $\mathcal{D}_c \subset \mathcal{D}$ 2. $|\mathcal{D}_c| \ll |\mathcal{D}|$ 3.

6.2

6.2.1 (Redundancy Score)

$$\mathcal{R}(x_i) = D(s_i) \cdot C(s_i) \cdot \left(1 - \max_m \text{SCX}_m(s_i)\right)$$

$$s_i = \arg \max_s \gamma_s(x_i)$$

$$- D(s_i) \rightarrow - C(s_i) \rightarrow - \max_m \text{SCX}_m(s_i) \rightarrow$$

$$\mathcal{R}_{\text{rel}}(x_i) = \frac{\mathcal{R}(x_i)}{\max_{x_j \in s_i} \mathcal{R}(x_j)}$$

6.2.2 Coreset

$$p_{\text{keep}}(x_i) = \min \left(1, \frac{n_{\text{keep}}(s_i) \cdot \text{Importance}(x_i)}{\sum_{x_j \in s_i} \text{Importance}(x_j)} \right)$$

$$\text{Importance}(x_i) = \frac{1}{1 + \mathcal{R}(x_i)}$$

$$n_{\text{keep}}(s) = \max \left(n_{\min}(s), |\mathcal{D} \cap s| \cdot [1 - D(s)] \cdot \frac{1}{1 + \bar{r}(s)} \right)$$

$$w_i = \frac{1}{p_{\text{keep}}(x_i)} \quad ()$$

$$w_i = \frac{|\mathcal{D} \cap s_i|}{|\mathcal{D}_c \cap s_i|} \cdot \frac{\text{Importance}(x_i)}{\sum_{x_j \in s_i} \text{Importance}(x_j)}$$

6.2.3 (Fidelity Constraints) 1

$$\left| \hat{R}_m(s; \mathcal{D}_c) - \hat{R}_m(s; \mathcal{D}) \right| < \delta_R, \quad \forall m, s$$

$$\hat{R}_m(s; \mathcal{D}_c) = \frac{\sum_{i \in \mathcal{D}_c \cap s} w_i \cdot \ell(f_m(x_i), y_i)}{\sum_{i \in \mathcal{D}_c \cap s} w_i}$$

2

$$|\rho(s; \mathcal{D}_c) - \rho(s; \mathcal{D})| < \delta_\rho, \quad \forall s$$

$$\rho(s; \mathcal{D}_c) = \frac{\sum_{i \in \mathcal{D}_c} \gamma_s(x_i) \cdot w_i}{\sum_{i \in \mathcal{D}_c} w_i}$$

3

$$|\mathcal{L}(\theta_c; \mathcal{D}_{\text{val}}) - \mathcal{L}(\theta; \mathcal{D}_{\text{val}})| < \delta_{\mathcal{L}}$$

$$\theta_c \ \mathcal{D}_c \ \theta \ \mathcal{D}$$

6.2.4 Anchor

$$\mathcal{B} = \{x_i \mid \text{BoundaryScore}(x_i) > \tau_{\text{boundary}}\}$$

$$\mathcal{D}_c \supseteq \mathcal{B} \quad ()$$

$$w_i = 1 + \text{BoundaryScore}(x_i)$$

$$\frac{|\mathcal{D}_c \cap \mathcal{B}|}{|\mathcal{B}|} = 1 \quad ()$$

6.2.5 Coreset Selection Coreset coreset

$$\left| \sum_{i \in \mathcal{D}} \ell(x_i; \theta) - \sum_{i \in \mathcal{D}_c} w_i \ell(x_i; \theta) \right| < \varepsilon, \quad \forall \theta \in \Theta$$

SCX-Compress Coreset

Coreset	SCX-Compress
//	+
	+ +
	anchor
	/

6.2.6 Dataset Distillation Dataset Distillation $\tilde{\mathcal{D}}$

$$\theta(\tilde{\mathcal{D}}) \approx \theta(\mathcal{D}), \quad \text{where } \theta(\mathcal{D}) = \arg \min_{\theta} \mathcal{L}(\mathcal{D}; \theta)$$

- SCX-Compress select existing data points - Dataset distillation generate new synthetic data points - SCX-Compress - SCX-Compress

6.3

Input:

- X: ndarray, shape=(N, d_X),
- y: ndarray, shape=(N,),
- gamma: ndarray, shape=(N, K), (1)
- R: ndarray, shape=(M, K), (2)
- SCX: ndarray, shape=(M, K), SCX (2)
- C: ndarray, shape=(K,), (3)
- D: ndarray, shape=(K,), (3)
- r_bar: ndarray, shape=(K,), (3)
- boundary_idx: list[int], (3)
- compression_rate: float, (0.5)
- delta_R: float, (0.05)
- delta_rho: float, (0.02)

Output:

- X_compressed: ndarray, shape=(N_c, d_X),
- y_compressed: ndarray, shape=(N_c,),
- weights: ndarray, shape=(N_c,),
- keep_mask: ndarray, shape=(N,), bool
- compression_stats: dict, :
 - compression_ratio_achieved: float
 - max_R_deviation: float
 - max_rho_deviation: float
 - boundary_preserved: bool

6.4

6SCX-Compress

Algorithm: SCXCompress(X, y, gamma, R, SCX, C, D, r_bar, boundary_idx, rate)

Step 1:

```
for i in 1..N:
    s_i = argmax_s gamma[i, :]
    R_i = D[s_i] * C[s_i] * (1 - max(SCX[:, s_i]))
    importance[i] = 1 / (1 + R_i)
```

Step 2:

```
for s in 1..K:
    n_s = count_members(X, gamma, s)
    n_keep[s] = max(
        max(n_boundary_in_s, 5),          # +
        n_s * (1 - D[s]) * (1 / (1 + r_bar[s] + eps))
    )
```

Step 3:

```
keep_mask = boundary_idx #
for s in 1..K:
    candidates = members_of_state(s) \ boundary_idx
    n_to_keep_in_state = n_keep[s] - count_boundary_in_state(s)
    prob = importance[candidates] / sum(importance[candidates])
    selected = sample_without_replacement(candidates, n_to_keep_in_state, prob)
    keep_mask[selected] = True
```

Step 4:

```
for s in 1..K:
    n_total_s = count_members(X, gamma, s)
    n_keep_s = sum(keep_mask & members_of_state(s))
    for i in members_of_state(s):
        if keep_mask[i]:
            weights[i] = n_total_s / n_keep_s * importance[i]
```

Step 5:

```
R_deviation = check_R_fidelity(R, X_compressed, y_compressed, weights, gamma)
rho_deviation = check_rho_fidelity(rho, X_compressed, weights, gamma)

while max(R_deviation) > delta_R or max(rho_deviation) > delta_rho:
    #
    s_violate = argmax(R_deviation)
    add_more_samples(s_violate, n_additional=5)
    recompute_weights()
    recompute_deviations()
```

Result: {X_compressed, y_compressed, weights, keep_mask, stats}

6.5

1	()	$\gamma_s(x),$
2	()	$R_m(s), \text{SCX}_m(s)$
3	()	$C(s), D(s), \bar{r}(s),$ boundary_idx
4	()	$V_{\text{remove}}(s)$

6.6

$O(N \cdot (K + M))$	$O(N)$
$O(K)$	$O(K)$
$O(N \log N)$	$O(N)$
$O(N_c \cdot M \cdot C_f)$	$O(M \cdot K)$
$O(N)$	$O(N)$

6.7

-	0.5
δ_R	0.05
δ_ρ	0.02
τ_{boundary}	0.3
n_{min}	5
-	5

7 (Active Learning Policy)

7.1

B SCX (point-wise) (state-wise)——””””

7.2

7.2.1 (Action Space) SCX

$$\mathcal{A} = \{\text{acquire, relabel, downweight, discard, route}\}$$

acquire	s	DFT
relabel	s oracle	
downweight	s	
discard	s	
route	s	

7.2.2 (Utility Function) -

$$U(a, s) = \underbrace{\Delta \mathcal{L}(a, s)} - \lambda_b \cdot \underbrace{\text{Cost}(a, s)}$$

acquire

$$U(\text{acquire}, s) = V_{\text{add}}(s) - \lambda_b \cdot \text{Cost}_{\text{label}}(s)$$

$$V_{\text{add}}(s) \propto \text{Cost}_{\text{label}}(s) \cdot s$$

relabel

$$U(\text{relabel}, s) = \bar{r}(s) \cdot \text{Conflict}(s) \cdot \rho(s) - \lambda_b \cdot \text{Cost}_{\text{relabel}}$$

$+$ $\rightarrow$

downweight

$$U(\text{downweight}, s) = N(s) \cdot \rho(s) \cdot \bar{r}(s) - \lambda_b \cdot \text{Cost}_{\text{downweight}}$$

$+$ $\rightarrow$

discard

$$U(\text{discard}, s) = N(s) \cdot (1 - \rho(s)) \cdot \bar{r}(s) - \lambda_b \cdot \text{Cost}_{\text{discard}}$$

$+ \rightarrow$

route

$$U(\text{route}, s) = \max_m \text{SCX}_m(s) \cdot \rho(s) \cdot \bar{r}(s) - \lambda_b \cdot \text{Cost}_{\text{route}}(m^*)$$

$\text{Conflict}(s)$ SCX

7.2.3 (Greedy)

$$(a^*, s^*) = \arg \max_{a,s} U(a, s)$$

(Proportional)

$$B(s) = B \cdot \frac{V_{\text{add}}(s)}{\sum_t V_{\text{add}}(t)}$$

s $U(a, s)$

Epsilon-

ε $1 - \varepsilon$

(s, a) Thompson sampling UCB

$$a^*(s) = \arg \max_a \left[\hat{U}(a, s) + \kappa \cdot \sqrt{\frac{2 \log t}{n_{a,s}}} \right]$$

t $n_{a,s}$ (a, s)

7.2.4 Point-wise State-wise Point-wise AL

$$x^* = \arg \max_{x \in \mathcal{U}} a_{\text{score}}(x, \mathcal{D}_L)$$

a_{score}

SCX State-wise AL

$$\text{Step 1 } () : s^* = \arg \max_s \max_a U(a, s)$$

$$\text{Step 2 } () : a^* = \arg \max_a U(a, s^*)$$

$$\text{Step 3 } () : \begin{cases} x^* = \arg \max_{x \in \mathcal{U} \cap s^*} \text{distance}(x, \mathcal{D}_L \cap s^*) & \text{if } a^* = \text{acquire} \\ x^* = \arg \max_{x \in \mathcal{D}_L \cap s^*} \text{Conflict}(x) & \text{if } a^* = \text{relabel} \end{cases}$$

State-wise

1. ""
2. $K \ll N \ O(K) \ O(N)$
- 3.
- 4.

7.2.5 DP-GEN DP-GEN (Deep Potential Generator)

$$\text{exploration : } x_{\text{new}} = \arg \max_x \sigma_{\text{model}}^2(x) \quad ()$$

exploitation :

	DP-GEN	SCX-AL
$V_{\text{add}}(s)$		
acquire/relabel/downweight/discard/route		
NoiseScore		
$R_m(s)$	$SCX_m(s)$	

7.2.6 D-optimality D-optimality $X \det(X^\top X)$

- D-optimality - SCX-AL - D-optimality - D-optimality SCX $D(s)$

7.2.7 DIRECT DIRECT (Diverse and Relevant Example selection)

- $\text{DIRECT } x^* = \arg \max [\lambda \cdot \text{diversity}(x) + (1 - \lambda) \cdot \text{relevance}(x)]$ - SCX-
AL $s^* = \arg \max_s V_{\text{add}}(s)$, s^* - DIRECT point-wise SCX-AL two-level state +
point - DIRECT relevance SCX-AL V_{add}

7.3

Input:

```
- X_labeled:      ndarray, shape=(N_L, d_X),
- y_labeled:      ndarray, shape=(N_L,),
- X_unlabeled:    ndarray, shape=(N_U, d_X),
- gamma_labeled:  ndarray, shape=(N_L, K),
- gamma_unlabeled: ndarray, shape=(N_U, K),
- V_add:          ndarray, shape=(K,), (4)
- R:              ndarray, shape=(M, K), (2)
- SCX:            ndarray, shape=(M, K), SCX (2)
- r_bar:          ndarray, shape=(K,), (3)
- rho:            ndarray, shape=(K,), (1/3)
- C:              ndarray, shape=(K,), (3)
- N:              ndarray, shape=(K,), (3)
- D:              ndarray, shape=(K,), (3)
- classification: dict, (3)
- budget:         float,
- costs:          dict,
- strategy:       str, 'greedy'/'proportional'/'epsilon_greedy'/'thompson'
- epsilon:        float, ( 0.1)
```

Output:

```
- selected_actions: list[tuple(state_id, action, budget_spent)]
- X_new:            ndarray, shape=(B_acquire, d_X),
- y_new:            ndarray, shape=(B_acquire,),
- relabel_indices:  list[int],
- downweight_indices: list[int],
```

- discard_indices: list[int],
- route_decisions: dict[sample_idx -> expert_id],
- remaining_budget: float,

7.4

7SCX

Algorithm: SCXActiveLearning(X_U , state_info, experts, budget, strategy)

```
: budget_remaining = budget
```

```
while budget_remaining > 0:
```

```
    Step 1: 1-4
```

```
    Step 2:  $U(a, s)$ 
```

```
    Step 3:  $(s^*, a^*)$ 
```

```
    if strategy == 'greedy':
```

```
         $(s^*, a^*) = \operatorname{argmax}_{\{a,s\}} U(a, s)$ 
```

```
    elif strategy == 'epsilon_greedy':
```

```
        if random() < epsilon:
```

```
             $(s^*, a^*) = \text{random}(s, a) \text{ from non-zero utility}$ 
```

```
        else:
```

```
             $(s^*, a^*) = \operatorname{argmax}_{\{a,s\}} U(a, s)$ 
```

```
    elif strategy == 'thompson':
```

```
         $(s^*, a^*) = \text{sample from posterior over } U(a, s)$ 
```

```
    Step 4:
```

```
    cost = costs[ $a^*$ ]
```

```
    if cost > budget_remaining:
```

```
        continue #
```

```
    if  $a^* == \text{'acquire'}$ :
```

```
        candidates =  $X_U$  intersect state  $s^*$ 
```

```
         $x_{\text{new}} = \text{select\_diverse\_representatives}(\text{candidates}, n=1)$ 
```

```
         $y_{\text{new}} = \text{query\_oracle}(x_{\text{new}})$  # oracle
```

```
         $X_L = X_L \cup \{(x_{\text{new}}, y_{\text{new}})\}$ 
```

```

        X_U = X_U \ {x_new}
    elif a* == 'relabel':
        candidates = X_L intersect state s*
        x_relabel = argmax Conflict(x) for x in candidates
        y_corrected = query_strong_oracle(x_relabel)
        update_label(x_relabel, y_corrected)
    elif a* == 'downweight':
        weight_reduction = C[s*] * (1 - N[s*])
        for x in X_L intersect state s*:
            sample_weight[x] *= weight_reduction
    elif a* == 'discard':
        for x in X_L intersect state s*:
            if NoiseScore(x) > tau_noise_extreme:
                X_L = X_L \ {x}
    elif a* == 'route':
        for x in X_L intersect state s*:
            m_best = argmax_m SCX[m, s*]
            label_replacement = f_{m_best}(x)
            #

    budget_remaining -= cost
    update_state_estimates() # r_bar, rho, D, L

```

Result: {selected_actions, X_L, sample_weights, budget_remaining}

7.5

1. : $\text{budget_remaining} \leq 0$
2. : $\max_{a,s} U(a, s) \leq 0$
3. :
4. : $D(s) > \tau_{D,\text{stop}} \quad L(s) < \tau_{L,\text{stop}}$

7.6

1 ()	$\gamma_s(x), \rho(s)$	
2 ()	$R_m(s), \text{SCX}_m(s)$	relabel

3	()	classification, $L(s)$, $N(s)$
4	()	$V_{\text{add}}(s)$, $V_{\text{remove}}(s)$

→ 2
→ 1

7.7

	$O(K \cdot M + K)$	$O(K \cdot \mathcal{A})$
	$O(K \cdot \mathcal{A})$	$O(K \cdot \mathcal{A})$
acquire	$O(N_U \cdot K)$	$O(N_U)$
	$O(N_L \cdot M \cdot C_f)$	$O(M \cdot K)$

7.8

	ε	0.1	epsilon-
	λ_b	1.0	
	$\tau_{D,\text{stop}}$	0.9	
	$\tau_{L,\text{stop}}$	0.05	
Thomspon	κ	0.5	UCB

8 Paper 1-3

8.1

SCX 4 SCX

8.2

()

Paper 1: ACE/PACE Expert Gauge + Merging

Paper 2: Residual-State Error Maps

Paper 3: Expert Compiler + Distillation

Paper 4: SCX (State-Conditioned eXpertise)

8.3 Paper 1: Gauge-Normalized Expert Merging $\rightarrow$ SCX Expert Reliability

8.3.1 Paper 1 Paper 1 ACE (Atomic Cluster Expansion) - ACE (gauge)
- ””

$$f_m(x) = \sum_i c_{m,i} \cdot B_i(x) \quad (\text{ACE})$$

Gauge transformation: $\tilde{c}_m = G \cdot c_m$

$G \tilde{c}_m$

8.3.2 SCX Paper 1

1. $\rightarrow R_m(s)$
2. RMSE $\rightarrow 1$
3. $\rightarrow$ SCX
4. $\rightarrow$

8.3.3 Paper 1

$$r_m(x) = |f_m^{\text{align}}(x) - f^*(x)|$$

Paper 1 sp^2 , sp^3 , SCX

$$\begin{aligned} \text{Paper 1 : } s(x) &\in \{\text{sp}^2, \text{sp}^3, \text{broken bond, surface}, \dots\} \\ \text{SCX : } s : \mathcal{X} &\rightarrow \{1, \dots, K\} \quad () \end{aligned}$$

Paper 1

$$E_{m,s} = \frac{1}{|\mathcal{D}_s|} \sum_{x_i \in \mathcal{D}_s} \ell(f_m(x_i), f^*(x_i))$$

SCX $R_m(s)$

8.4 Paper 2: Residual-State Error Maps $\rightarrow$ SCX State Discovery

8.4.1 Paper 2 Paper 2 (Residual-State Maps)

1. x_i $r_i = \ell(f(x_i), y_i)$ 2. s $\bar{r}(s) = \mathbb{E}[r_i | x_i \in s]$ 3. $\bar{r}(s)$

8.4.2 SCX Paper 2

1. $\rightarrow$
2. Paper 2 ”” $\rightarrow 2$
3. $\rightarrow$
4. $\rightarrow V_{\text{add}}(s)$ $r(s)$

8.4.3 Paper 2

$$\mathcal{R}(s) = \{(m, \bar{r}_m(s)) \mid m = 1, \dots, M\}$$

SCX

$$R_m(s) = \mathbb{E}_{x \sim P(\cdot | s)}[\ell(f_m(x), f^*(x))]$$

Paper 2 PCA/UMAP SCX 1

8.5 Paper 3: Expert Compiler Distillation $\rightarrow$ SCX Expert Routing + Teacher Generation

8.5.1 Paper 3 Paper 3 (Expert Compiler) 1. 2. / 3.

8.5.2 SCX Paper 3

1. Paper 3 $\rightarrow$ 5
2. $\rightarrow$
3. $\rightarrow w_m(x)$
4. $\rightarrow$

8.5.3 Paper 3 SCX

$$\text{Paper 3 : } m^*(x) = \arg \min_m \sum_s \gamma_s(x) \cdot \underbrace{\text{validation_err}(m, s)}_{R_m(s)}$$

Paper 3

$$\mathcal{L}_{\text{distill}}(x) = \sum_m w_m(x) \cdot \ell(f_{\text{student}}(x), f_m(x))$$

$w_m(x) \rightarrow$ SCX

8.6

8.6.1

SCX	Paper 1	Paper 2	Paper 3
f_m	ACE $f_1, f_2, \dots$		
s			
$R_m(s)$	RMSE		
$\text{SCX}_m(s)$	$<$	$()$	
$V_{\text{add}}(s)$		$() +$	
$m^*(x)$	$()$	$()$	
$\mathcal{A}$	DFT		$++$

8.6.2

Paper 1: " ACE "
 $\downarrow$
 Paper 2: ""
 $\downarrow$
 Paper 3: ""
 $\downarrow$
 Paper 4: " + "

8.6.3 ML

1. Paper 1 2. Paper 2 3. Paper 3 4. Paper 4 (SCX)

Paper 1
 $\rightarrow E_{\{m,s\}}$
 $\rightarrow$ Paper 2
 $\rightarrow$
 $\rightarrow$ Paper 3
 $\rightarrow$
 $\rightarrow$ Paper 4 SCX

8.7

1. SCX Paper 3 V_{add}
2. Paper 1-3 SCX
3. SCX MLIP NLP
4. Paper 3 SCX SCX

8.8

		SCX
Paper 1	$O(N \cdot C_{\text{ACE}} \cdot M)$	$R_m(s)$
Paper 2	$O(N \cdot (d^2 + K \cdot M))$	
Paper 3	$O(N \cdot M \cdot C_f + N_{\text{distill}} \cdot C_{\text{student}})$	
Paper 4	$O(N \cdot M \cdot C_f + N \cdot K \cdot M)$	

SCX: State-Conditioned eXpertise

-

Raw Data X
(//
/)

1 (State Discovery & Representation)

$X \rightarrow \begin{matrix} \text{PCA/UMAP} & \text{GMM/} & (s), _s, _s \\ \text{ACE/SOAP} & \rightarrow & \text{K-means} & B(x) \end{matrix}$

2 $_s(x) \ (s)$
(Expert Reliability Estimation)

$R_m(s)$ 3
() (Data Four-
Classification)

Disagreement
 $R_m(s)$

$C(s) \ N(s) \ D(s)$

/

$L(s) =$
 $C(1-N)$

Output: $R_m(s)$, $SCX_m(s)$
 (shape: $M \times K$)
 Valuable/

:

Redundant/
 Noisy/
 Expert-dependent

5
 (Expert Routing)

4

$m^*(x)$ $w_m(x)$

(State Data
 Valuation)

$V_{add}(s) =$

$rL(1-D)$
 $\max_m \ SCX_m(s)$

Conflict /

$V_{remove}(s) =$

$rD(1-)$

Output: y_{pred} , $w_m(x)$

6SCX-Compress ()

Coreset
 $R(x_i) +$

Output: D_c , weights, keep_mask

7 (Active Learning Policy)

$U(a, s)$
 $U(\text{acquire}) \quad U(\text{relabel}) \quad U(\text{downweight}) \quad U(\text{discard}) \quad U(\text{route})$

(Greedy / Thompson / Epsilon-greedy)
 $\rightarrow (s^*, a^*) \rightarrow \rightarrow$

:
- (acquire)
- (relabel)
- / (downweight/discard)
- (route-to-expert)

(Model Update)
/ /

(Loop to Module 1)

Paper 1-3 (8)

Paper 1 Paper 2 Paper 3 Paper 4 (SCX)
 → → →
 ()

: f_m → ACE; s → ; _s(x) →
 R_m(s) → RMSE; V_add → DFT

X ()

[1] _s(x), (s),

[3] C(s), N(s), D(s), L(s)

[4] V_add(s), V_remove(s)

[2] R_m(s), SCX_m(s)

[5] /

[7] (s*, a*)

[6] ()

[1] ()

SCX

1. 2. 3. 4. 5. 6. 7. 8.

SCX ———+